

Aplicação Python para Análise de Documentos PDF com IA

Análise do Escopo Desenvolvido (Versão 0.2)

O trabalho realizado superou significativamente o planejamento inicial para a Fase 1 (MVP), incorporando funcionalidades previstas para fases posteriores e adicionando uma robustez notável à aplicação.

Funcionalidades Implementadas Além do MVP Original

- **Exportação por Template (v1.0):** O `doc_generator.py` já implementa a exportação para templates `.docx`, uma funcionalidade planejada para a Fase 2 (Objetivo 2.4).
- **Suporte Estrutural a Múltiplos LLMs (v1.0):** A arquitetura, especialmente em `llm_settings_view.py` e `ai_orchestrator.py`, já está preparada para gerenciar múltiplos provedores e modelos, incluindo o armazenamento de chaves de API distintas por serviço. Isso adianta grande parte do Objetivo 2.1.
- **Gerenciamento Avançado de Prompts (v1.0):** A aplicação já carrega prompts do Firestore e implementa uma lógica de prompts segmentados (`PROMPTS_SEGMENTADOS_for_INITIAL_ANALYSIS`), o que é muito mais avançado do que o "prompt único" planejado e se alinha ao Objetivo 2.3. A utilização de modelos Pydantic para saídas estruturadas também é um grande avanço.
- **Interface de Usuário (UI) Completa e Robusta:** A UI Flet implementada é muito mais do que um MVP. Inclui:
 - Gerenciamento completo de perfil de usuário (`profile_view.py`).
 - Views dedicadas para todas as configurações (`llm_settings_view.py`, `proxy_settings_view.py`).
 - Um rico conjunto de componentes reutilizáveis (`components.py`).
 - Fluxo de autenticação completo com verificação de email e redefinição de senha.
 - Mecanismo de atualização de token proativo para manter a sessão do usuário ativa.
- **Módulo-Agente de Análise de NC:** A `nc_analyze_view.py` funciona como um agente de IA completo e especializado, com um fluxo de trabalho claro (carregar -> processar -> analisar -> exportar) e uma interface rica para edição e feedback, superando a expectativa de "exibir resultados em um TextField".
- **Métricas de Feedback Detalhadas:** O sistema de feedback implementado é sofisticado, calculando similaridade de texto e coletando dados granulares sobre as edições do usuário, alinhado ao Objetivo 1.5 mas com maior profundidade.
- **Verificação de Versão da Aplicação:** A função `check_app_version` em `utils.py` já estabelece a base para o sistema de atualização automática da Fase 3 (Objetivo 3.2).

Funcionalidades Planejadas e ainda Não Implementadas (Próximas Fases)

- **RAG e Chat com PDF (v1.0):** O núcleo da Fase 2 ainda está pendente, como planejado. O placeholder `others_view.py` está corretamente no lugar.
- **Anonimização de Dados (v1.5):** O `anonymizer.py` é um esqueleto, como planejado para a Fase 3.

- Atualizador Automático (v1.5): A lógica do Updater.exe ainda não foi desenvolvida, conforme o plano da Fase 3.
- Integração RPA e LLMs Locais: Funcionalidades das fases 4 e 5 estão corretamente pendentes.

FASES:

****FASE 1: Versão 0.2 – MVP Completo (ESTADO ATUAL)****

Objetivo: Entregar um Módulo-Agente de IA especializado, funcional e distribuível, com interface de usuário completa e infraestrutura de backend robusta.

* ****Análise de PDF:**** Pipeline completo de extração, classificação, agrupamento e agregação de texto.

* ****Análise com LLM:**** Integração com LLM (OpenAI) para análise estruturada de documentos, utilizando um sistema de prompts segmentados carregados da nuvem.

* ****Interface Flet Completa:**** UI com autenticação de usuário (login, signup, verificação de email, reset de senha), gerenciamento de perfil, configurações de proxy e LLM, e um fluxo de trabalho guiado para análise de documentos.

* ****Gerenciamento Seguro de Chaves:**** Armazenamento seguro de chaves de serviço e chaves de API do usuário usando Keyring e criptografia.

* ****Exportação Avançada:**** Funcionalidade de exportação para DOCX, tanto simples quanto baseada em templates customizáveis pelo usuário.

* ****Logging e Métricas:**** Sistema robusto de logging local e em nuvem (específico do usuário) e coleta detalhada de métricas de uso e feedback de precisão da IA.

****FASE 2: Versão 1.0 – RAG e Interação Avançada****

Objetivo: Implementar Retrieval Augmented Generation (RAG) para melhorar a contextualização e permitir interação conversacional com documentos.

* Implementar RAG "on the fly" para indexação vetorial de documentos em memória.

* Aprimorar a análise de documentos longos injetando contexto relevante no prompt.

* Desenvolver a interface "Chat com seu PDF" para permitir perguntas e respostas em linguagem natural sobre o documento carregado.

****FASE 3: Versão 1.5 – Segurança e Atualizações****

Objetivo: Incrementar a segurança dos dados processados e facilitar a distribuição de novas versões da aplicação.

* Implementar sistema de anonimização de dados sensíveis (NER) antes do envio para APIs externas.

* Desenvolver e integrar um atualizador automático (` Updater.exe `).

****FASE 4: Versão 2.0 – Integração Externa (RPA)****

Objetivo: Permitir que processos de RPA interajam com as funcionalidades da aplicação.

* Desenvolver e documentar uma interface (API HTTP local ou Firebase Functions) para integração com scripts de RPA.

****FASE 5: Versão 2.5 – Processamento Offline (LLMs Locais)****

Objetivo: Oferecer a opção de processamento totalmente offline, utilizando LLMs executadas localmente.

* Integrar suporte para execução de LLMs locais (via Llama.cpp, Ollama, ou similar).

* Desenvolver interface para gerenciamento e seleção de modelos locais.

****FASE 6: Versão 3.0 – Arquitetura de Agentes (MCP)****

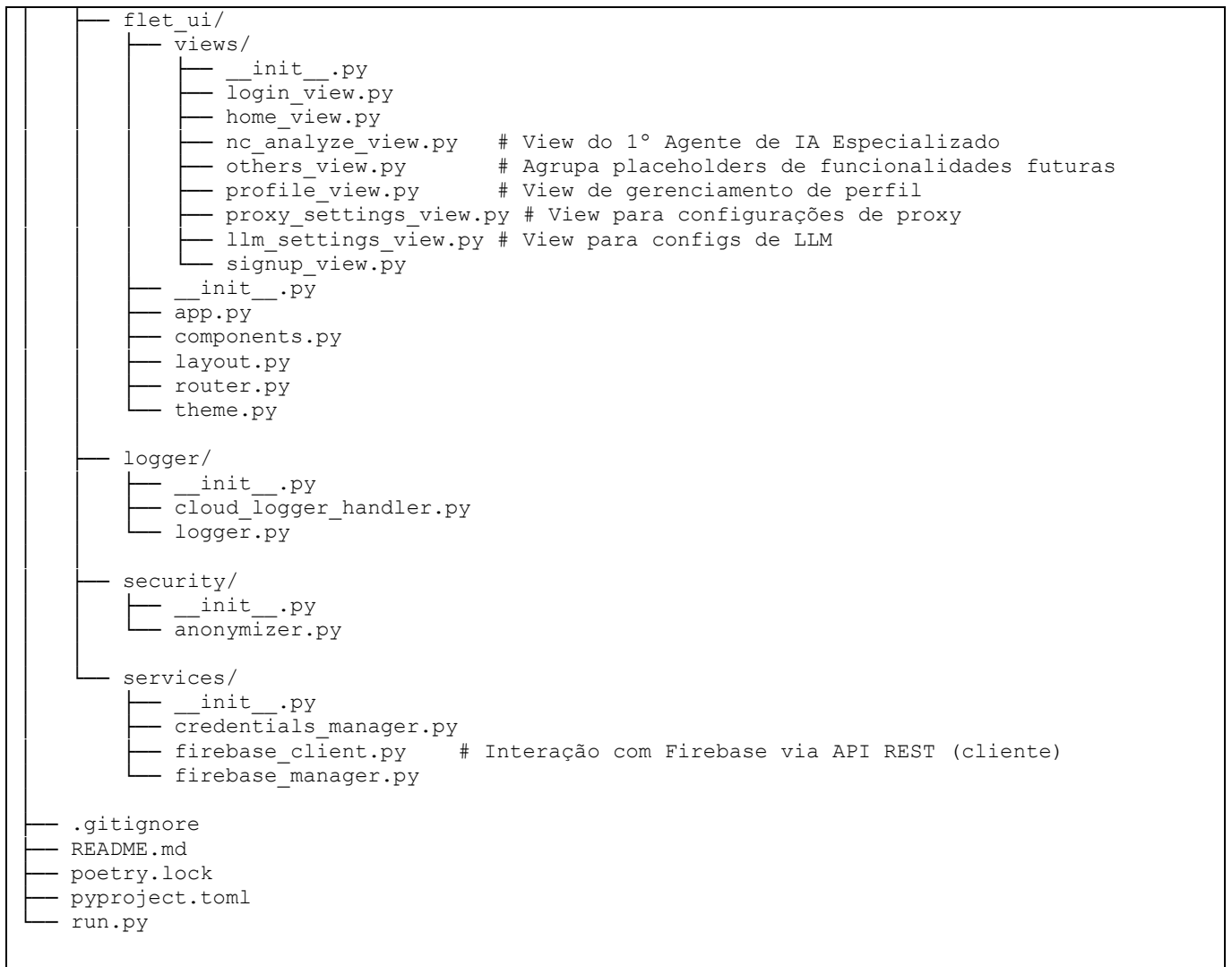
Objetivo: Evoluir a arquitetura para um modelo de múltiplos agentes especializados, permitindo maior autonomia e flexibilidade.

* Re-arquitetar a lógica de IA utilizando um pipeline de agentes (Extração, RAG, Análise, Formatação).

* Implementar a comunicação e troca de contexto entre agentes seguindo os princípios do Model Context Protocol (MCP).

Plano Operacional Detalhado

```
docs_analyzer_3/
├── .git/
├── assets/
│   ├── templates_docx/
│   │   └── *.docx
│   └── *.png
├── logs/
│   └── *.log
├── src/
│   ├── __init__.py
│   ├── config_manager.py
│   ├── settings.py
│   ├── utils.py
│   └── core/
│       ├── __init__.py
│       ├── ai_orchestrator.py
│       ├── doc_generator.py
│       ├── pdf_processor.py
│       └── prompts.py
```



Registros de Desenvolvimento

Troubleshooting Log (Issue Resolution Log)

...

Guia para Compilação de Aplicações Flet com PyInstaller

1. Metodologia de Compilação e Depuração

- * **Problema:** A compilação é muito lenta (devido a libs como PyTorch/SentenceTransformers), tornando a depuração iterativa inviável.
- * **Causa:** PyInstaller reanalisa todas as dependências a cada build limpa, o que é demorado.
- * **Solução:** Crie dois arquivos `.spec`: `app-debug.spec` (com `console=True`, `upx=False`) para testes rápidos e `app.spec` (com `console=False`, `upx=True`) para a build final de produção.
- * **Dica:** Para depurar, compile sem a flag `--clean` e execute o `.exe` a partir de um terminal (CMD/PowerShell) para capturar os `tracebacks` de erro.

2. Gerenciamento de Caminhos para Assets

- * **Problema:** O código não encontra arquivos de `assets` (imagens, modelos de ML, templates) após a compilação.
- * **Causa:** O caminho base da aplicação muda. Em desenvolvimento é a raiz do projeto, mas no executável (`frozen`), passa a ser o diretório do `.exe`.
- * **Solução:** Crie e utilize uma função `get_resource_path` que verifica `if getattr(sys, 'frozen', False)` e usa `os.path.dirname(sys.executable)` como o caminho base no ambiente compilado.
- * **Observação:** Não use esta função para assets da UI do Flet (como `ft.Image.src`). Registre a pasta `assets` no `ft.app()` e use caminhos relativos (ex: `/logo.png`).

3. Carregamento de Modelos de ML (ex: SentenceTransformers)

- * **Problema:** A aplicação congela ou exibe uma tela branca ao iniciar, apenas na versão compilada.
- * **Causa:** A primeira importação ou instanciação de bibliotecas pesadas como `sentence_transformers` bloqueia a thread principal da GUI por vários segundos.
- * **Solução:** No `run.py`, inicie uma thread de background (`threading.Thread`) que realiza a importação e instanciação do modelo, armazenando o objeto pronto em um módulo de cache global (ex: `src/app_cache.py`).
- * **Observação:** Use um `threading.Event` para sincronização, garantindo que qualquer função que precise do modelo possa esperar (`event.wait()`) caso o pré-carregamento ainda não tenha terminado.

4. Erro `AttributeError: ... isatty` em Aplicações sem Console

- * **Problema:** A aplicação compilada com `--noconsole` (`console=False`) falha ao iniciar com um `AttributeError` relacionado a `isatty` na biblioteca `uvicorn`.
- * **Causa:** Em um ambiente sem console, `sys.stdout` é `None`. O servidor web do Flet (Uvicorn) trava ao tentar chamar `sys.stdout.isatty()`.
- * **Solução:** No `run.py`, "remende" (`monkey-patch`) `sys.stdout` e `sys.stderr` com uma classe `DummyStream` que possui um método `isatty()` que sempre retorna `False`.
- * **Observação:** Aplique este patch condicionalmente, apenas quando `getattr(sys, 'frozen', False)` for verdadeiro e `sys.stdout` for `None`, e faça isso antes da chamada `ft.app()`.

5. Download de Arquivos na Versão Web (Exportação DOCX)

- * **Problema:** Usar `page.launch_url` para iniciar um download de um arquivo local reinicia a aplicação ou não funciona.

- * **Causa:** O roteador do Flet intercepta a URL do arquivo como uma rota de navegação, causando comportamento inesperado.
- * **Solução:** Use o controle `ft.FilePicker``. Gere o conteúdo do arquivo em memória (bytes) no backend e passe-o para `FilePicker.save_file()`, que aciona um diálogo "Salvar Como" nativo do navegador.
- * **Observação:** Essa abordagem é mais robusta, segura e melhora a experiência do usuário, eliminando a necessidade de salvar arquivos temporários para download.

6. Configuração do Arquivo `.spec``

- * **Problema:** PyInstaller não encontra módulos (imports dinâmicos, namespaces) ou arquivos de dados (.token, modelos de ML) e a pasta `assets`` é colocada no lugar errado (`_internal``).
- * **Causa:** Limitações da análise estática do PyInstaller.
- * **Solução:** Edite o arquivo `.spec`` manualmente. Use o argumento `hiddenimports`` para módulos ausentes e `datas`` para incluir arquivos e diretórios de dados.
- * **Observação:** Para incluir diretórios inteiros (como `assets`` e `flet_web/web``), a sintaxe `datas=[('caminho/origem', 'caminho/destino')]` é a forma moderna e recomendada, substituindo o antigo `Tree``.

Exemplo de Configuração `.spec`` Essencial:

```
` `` python
# no seu IAnalista.spec
a = Analysis(
    ['run.py'],
    datas=[
        # Garante que a pasta assets fique na raiz do app compilado
        ('assets', 'assets'),
        # Inclui os assets web essenciais do Flet
        (r'C:\caminho\para\seu\venv\Lib\site-packages\flet_web\web', 'web'),
        # Inclui os arquivos de encoding do tiktoken
        (r'C:\caminho\para\seu\venv\Lib\site-packages\tiktoken_ext', 'tiktoken_ext')
    ],
    hiddenimports=[
        'keyring.backends.Windows',
        'src.flet_ui.views.login_view',
        'src.flet_ui.views.signup_view',
        'src.flet_ui.views.home_view',
        'src.flet_ui.views.profile_view',
        'src.flet_ui.views.proxy_settings_view',
        'src.flet_ui.views.llm_settings_view',
        'src.flet_ui.views.nc_analyze_view',
        'src.flet_ui.views.others_view',
        'unidecode',
        'sentence_transformers',
        'tiktoken_ext.cl100k_base',
        'jaraco.text',
    ],
    ...
)
```

Problema do PyInstaller 6+: Alternativas para Colocar Arquivos de Dados no Diretório Raiz

O PyInstaller 6+ introduziu uma mudança significativa na estrutura de distribuição onedir, onde todos os arquivos de dados especificados na diretiva `datas=` são agora colocados na subpasta `_internal` em vez do diretório raiz da distribuição junto ao executável. Esta mudança quebra aplicações que esperam encontrar seus recursos no mesmo nível do executável.

1. Solução Via Script Python Pós-Build

A técnica mais robusta e recomendada pelos desenvolvedores do PyInstaller é usar código Python personalizado no final do arquivo `spec` para copiar os arquivos necessários após a construção:

```
...
```

```
# -*- mode: python ; coding: utf-8 -*-
```

```
import shutil
```

```
import os
```

```
# Configuração padrão do spec file
```

```
a = Analysis(  
    ['main.py'],  
    pathex=[],  
    binaries=[],  
    datas=[('assets', 'assets')], # Isso irá para _internal/assets  
    hiddenimports=[],  
    # ... outras configurações  
)
```

```
pyz = PYZ(a.pure, a.zipped_data, cipher=None)
```

```
exe = EXE(  
    pyz,  
    a.scripts,  
    exclude_binaries=True,  
    name='meu_app',  
    debug=False,  
    bootloader_ignore_signals=False,  
    strip=False,  
    upx=True,  
    console=True,  
)
```

```
coll = COLLECT(  
    exe,  
    a.binaries,  
    a.zipfiles,  
    a.datas,  
    strip=False,  
    upx=True,  
    upx_exclude=[],  
    name='meu_app',
```

)

CÓDIGO PERSONALIZADO PÓS-BUILD

Copia a pasta assets da _internal para o diretório raiz

dist_path = os.path.join(DISTPATH, 'meu_app')

internal_assets = os.path.join(dist_path, '_internal', 'assets')

root_assets = os.path.join(dist_path, 'assets')

if os.path.exists(internal_assets):

if os.path.exists(root_assets):

shutil.rmtree(root_assets)

shutil.copytree(internal_assets, root_assets)

print(f"Assets copiados para: {root_assets}")

```\n

---